

```

# This Python 3 environment comes with many helpful analytics
libraries installed
# It is defined by the kaggle/python Docker image:
https://github.com/kaggle/docker-python
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/"
directory
# For example, running this (by clicking run or pressing Shift+Enter)
will list all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/)
that gets preserved as output when you create a version using "Save &
Run All"
# You can also write temporary files to /kaggle/temp/, but they won't
be saved outside of the current session

/kaggle/input/datasets/kyanyoga/sample-sales-data/
sales_data_sample.csv

```

1. IMPORT LIBRARIES

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

sns.set(style="whitegrid")

```

2. LOAD DATA

```

df =
pd.read_csv("/kaggle/input/datasets/kyanyoga/sample-sales-data/sales_d
ata_sample.csv", encoding='latin1')

df.head()

```

	ORDERNUMBER	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	
SALES \					
0	10107	30	95.70	2	2871.00
1	10121	34	81.35	5	2765.90
2	10134	41	94.74	2	3884.34
3	10145	45	83.26	6	3746.70
4	10159	49	100.00	14	5205.27

	ORDERDATE	STATUS	QTR_ID	MONTH_ID	YEAR_ID	...	\
0	2/24/2003 0:00	Shipped	1	2	2003	...	
1	5/7/2003 0:00	Shipped	2	5	2003	...	
2	7/1/2003 0:00	Shipped	3	7	2003	...	
3	8/25/2003 0:00	Shipped	3	8	2003	...	
4	10/10/2003 0:00	Shipped	4	10	2003	...	

	ADDRESSLINE1	ADDRESSLINE2	CITY	STATE	\
0	897 Long Airport Avenue	NaN	NYC	NY	
1	59 rue de l'Abbaye	NaN	Reims	NaN	
2	27 rue du Colonel Pierre Avia	NaN	Paris	NaN	
3	78934 Hillside Dr.	NaN	Pasadena	CA	
4	7734 Strong St.	NaN	San Francisco	CA	

	POSTALCODE	COUNTRY	TERRITORY	CONTACTLASTNAME	CONTACTFIRSTNAME
DEALSIZE					
0	10022	USA	NaN	Yu	Kwai
Small					
1	51100	France	EMEA	Henriot	Paul
Small					
2	75508	France	EMEA	Da Cunha	Daniel
Medium					
3	90003	USA	NaN	Young	Julie
Medium					
4	NaN	USA	NaN	Brown	Julie
Medium					

[5 rows x 25 columns]

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2823 entries, 0 to 2822
Data columns (total 25 columns):
#   Column                Non-Null Count  Dtype
---  -
0   ORDERNUMBER           2823 non-null  int64
```

1	QUANTITYORDERED	2823	non-null	int64
2	PRICEEACH	2823	non-null	float64
3	ORDERLINENUMBER	2823	non-null	int64
4	SALES	2823	non-null	float64
5	ORDERDATE	2823	non-null	object
6	STATUS	2823	non-null	object
7	QTR_ID	2823	non-null	int64
8	MONTH_ID	2823	non-null	int64
9	YEAR_ID	2823	non-null	int64
10	PRODUCTLINE	2823	non-null	object
11	MSRP	2823	non-null	int64
12	PRODUCTCODE	2823	non-null	object
13	CUSTOMERNAME	2823	non-null	object
14	PHONE	2823	non-null	object
15	ADDRESSLINE1	2823	non-null	object
16	ADDRESSLINE2	302	non-null	object
17	CITY	2823	non-null	object
18	STATE	1337	non-null	object
19	POSTALCODE	2747	non-null	object
20	COUNTRY	2823	non-null	object
21	TERRITORY	1749	non-null	object
22	CONTACTLASTNAME	2823	non-null	object
23	CONTACTFIRSTNAME	2823	non-null	object
24	DEALSIZE	2823	non-null	object

dtypes: float64(2), int64(7), object(16)

memory usage: 551.5+ KB

df.isnull().sum()

ORDERNUMBER	0
QUANTITYORDERED	0
PRICEEACH	0
ORDERLINENUMBER	0
SALES	0
ORDERDATE	0
STATUS	0
QTR_ID	0
MONTH_ID	0
YEAR_ID	0
PRODUCTLINE	0
MSRP	0
PRODUCTCODE	0
CUSTOMERNAME	0
PHONE	0
ADDRESSLINE1	0
ADDRESSLINE2	2521
CITY	0
STATE	1486
POSTALCODE	76
COUNTRY	0

```
TERRITORY          1074
CONTACTLASTNAME    0
CONTACTFIRSTNAME   0
DEALSIZE           0
dtype: int64
```

3. DATA CLEANING

```
# Drop unnecessary columns
drop_cols = [
    'ADDRESSLINE1', 'ADDRESSLINE2', 'CONTACTFIRSTNAME',
    'CONTACTLASTNAME', 'POSTALCODE', 'ORDERNUMBER'
]
df.drop(columns=drop_cols, inplace=True)

# Handle missing values
df['STATE'] = df['STATE'].fillna('Unknown')
df['TERRITORY'] = df['TERRITORY'].fillna('Unknown')

# Convert date
df['ORDERDATE'] = pd.to_datetime(df['ORDERDATE'])
```

4. FEATURE ENGINEERING

```
df['YEAR'] = df['ORDERDATE'].dt.year
df['MONTH'] = df['ORDERDATE'].dt.month

df.drop('ORDERDATE', axis=1, inplace=True)
```

5. ENCODING

```
le = LabelEncoder()

for col in df.select_dtypes(include='object').columns:
    df[col] = le.fit_transform(df[col])

X = df.drop('STATUS', axis=1)
y = df['STATUS']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

6. DECISION TREE MODEL

```
dt = DecisionTreeClassifier(criterion='entropy', max_depth=4,
    random_state=42)
dt.fit(X_train, y_train)
```

```
DecisionTreeClassifier(criterion='entropy', max_depth=4,  
random_state=42)
```

```
y_pred = dt.predict(X_test)
```

7. EVALUATION

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
print("\nClassification Report:\n")  
print(classification_report(y_test, y_pred))
```

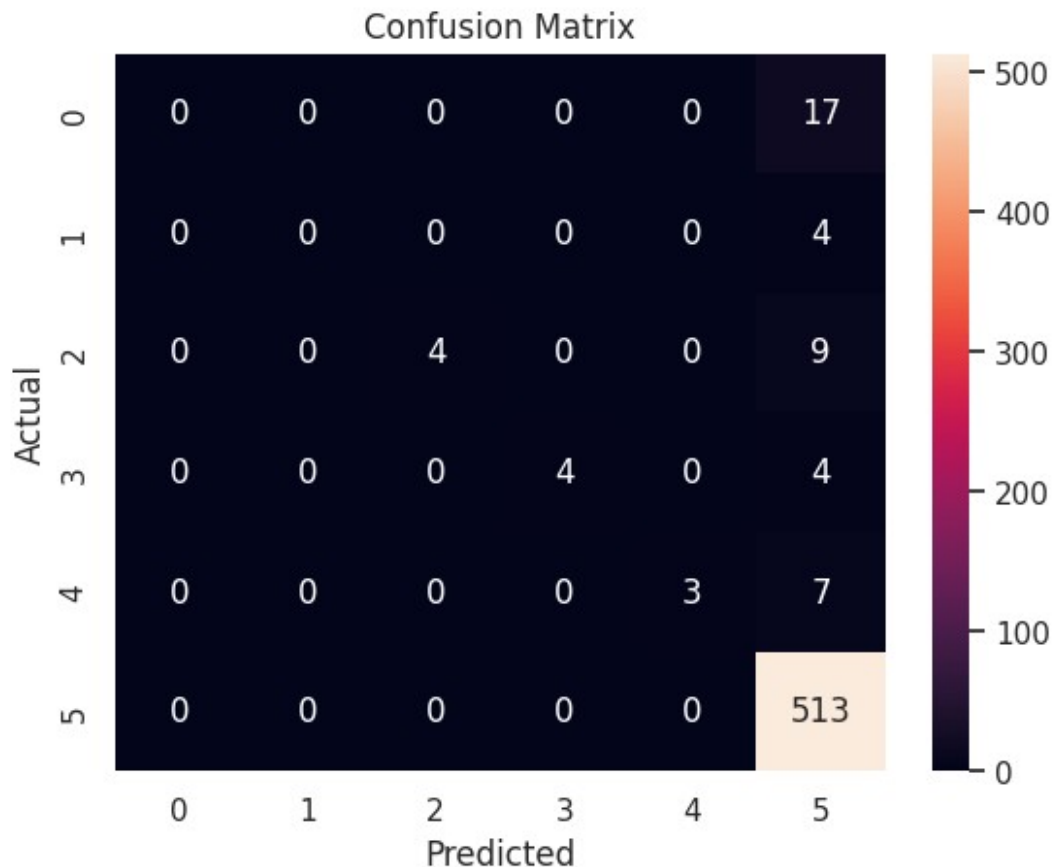
Accuracy: 0.9274336283185841

Classification Report:

	precision	recall	f1-score	support
0	0.00	0.00	0.00	17
1	0.00	0.00	0.00	4
2	1.00	0.31	0.47	13
3	1.00	0.50	0.67	8
4	1.00	0.30	0.46	10
5	0.93	1.00	0.96	513
accuracy			0.93	565
macro avg	0.65	0.35	0.43	565
weighted avg	0.90	0.93	0.90	565

```
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/  
_classification.py:1565: UndefinedMetricWarning: Precision is ill-  
defined and being set to 0.0 in labels with no predicted samples. Use  
'zero_division' parameter to control this behavior.  
  _warn_prf(average, modifier, f"{metric.capitalize()} is",  
len(result))  
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classificatio  
n.py:1565: UndefinedMetricWarning: Precision is ill-defined and being  
set to 0.0 in labels with no predicted samples. Use 'zero_division'  
parameter to control this behavior.  
  _warn_prf(average, modifier, f"{metric.capitalize()} is",  
len(result))  
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classificatio  
n.py:1565: UndefinedMetricWarning: Precision is ill-defined and being  
set to 0.0 in labels with no predicted samples. Use 'zero_division'  
parameter to control this behavior.  
  _warn_prf(average, modifier, f"{metric.capitalize()} is",  
len(result))
```

```
# Confusion Matrix
plt.figure()
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d')
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



8. DECISION TREE VISUALIZATION

```
plt.figure(figsize=(25,12))

plot_tree(
    dt,
    feature_names=X.columns,
    class_names=[str(i) for i in np.unique(y)],
    filled=True,
    rounded=True,
    fontsize=10
)
```

```
plt.title("Decision Tree Visualization")
plt.show()
```

Decision Tree Visualization

